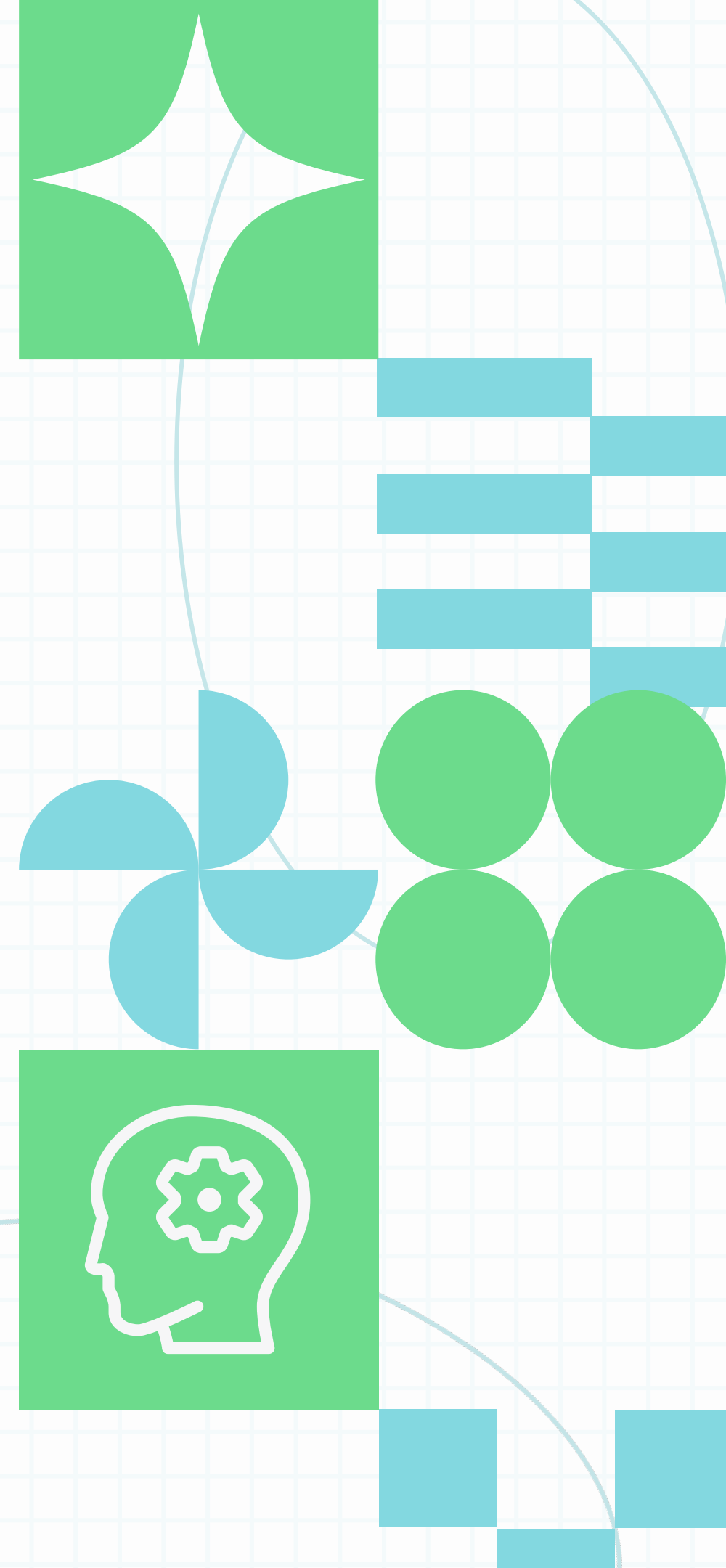


Attention & Power Retention

지피다 연구원 김재현

E. jh.kim@zipida.com | T. 010-4744-0949





목차



Transformer

Attention / Retention

Complexity

Code Implement

Architecture

Encoder

- Input Embedding
 $[B, N] \rightarrow [B, N, D]$
- Positional Encoding
- Multi-Head Attention
 $q = k = v [B, N, D]$ ($D = H * C$)
Head $[B, H, N, C] \rightarrow [B, N, D]$
- Add LayerNormalization
- FeedForward Network
- KV Cache
Attention Score
 $[B, H, 1, C] @ [B, H, C, t] \rightarrow [B, H, 1, t]$

Decoder

- Shifted Right Input Embedding
 $[B, N] \rightarrow [B, N, D]$
- Positional Encoding
- Masked Multi-Head Attention
Causal Mask $[B, 1, N, N]$
Attention Score
 $[B, H, N, C] @ [B, H, C, N] \rightarrow [B, H, N, N]$
Output $[B, N, D]$
- Add LayerNormalization
- Multi-Head Cross Attention
- FeedForward Network

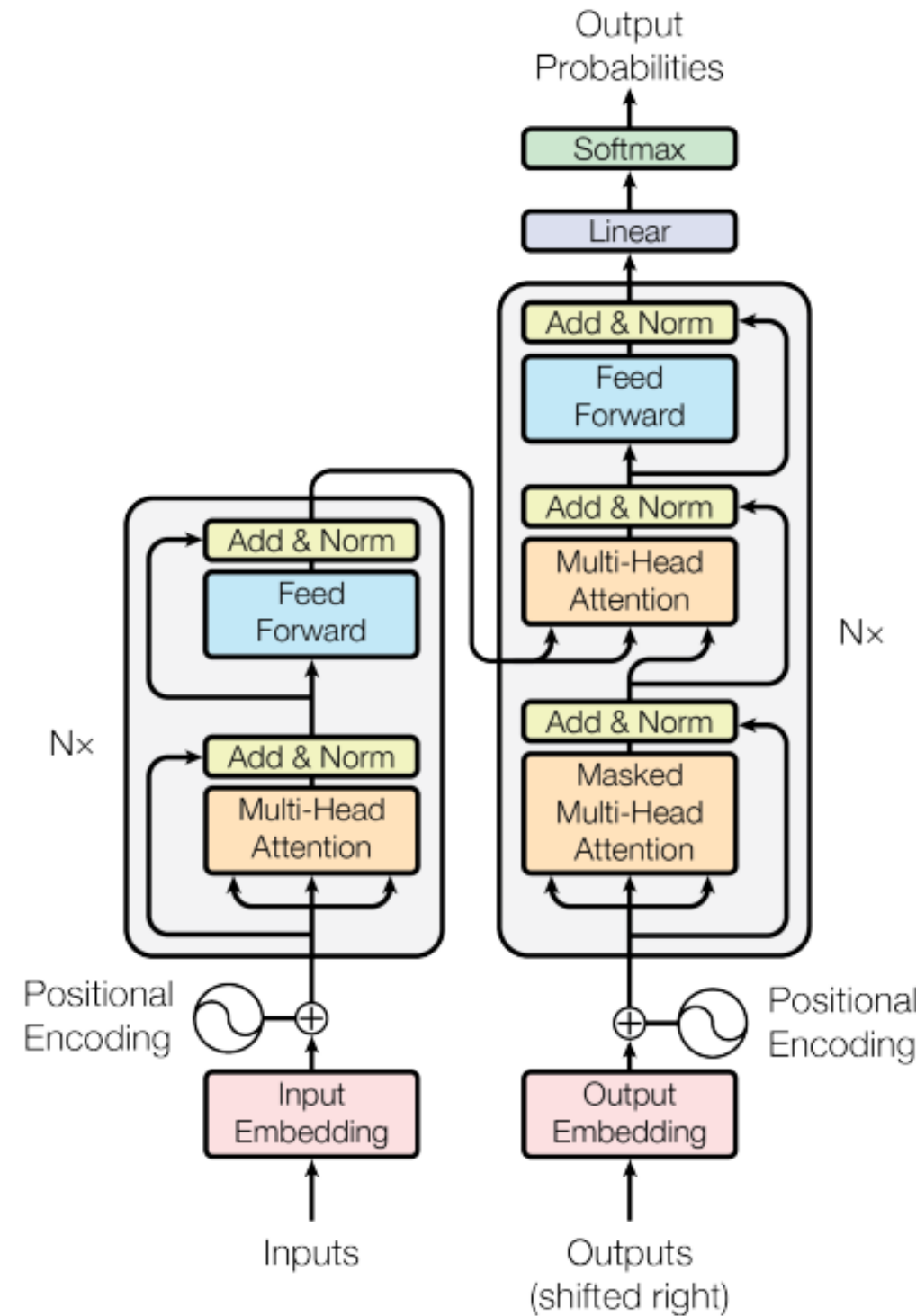


Figure 1: The Transformer - model architecture.

Encoder

‘ 피곤하냐 ’ : [1.1, 0.1, 0.0, 1.0]

불필요한 정보는 버려지고 핵심적인 정보만 강화

모델 차원 : 4 → 16 → 4

Q(Query) : ‘ 피곤하냐 ’ 는 다른 단어랑 어떤 관계?

K(Key) : 다른 단어들의 문맥적 특성

V(Value) : 다른 단어들의 실제 의미 정보

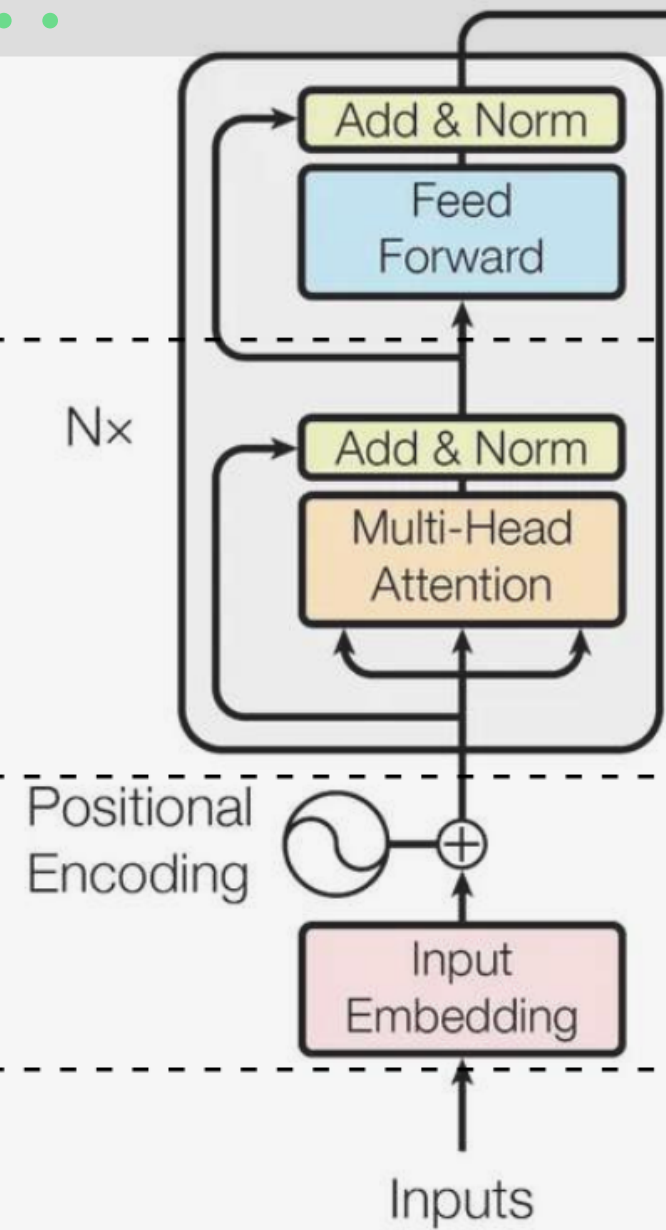
‘피곤하냐’ 벡터는 단순한 '피로'를 넘어
'잠을 자고 싶을 만큼의 피로'라는 문맥을 흡수

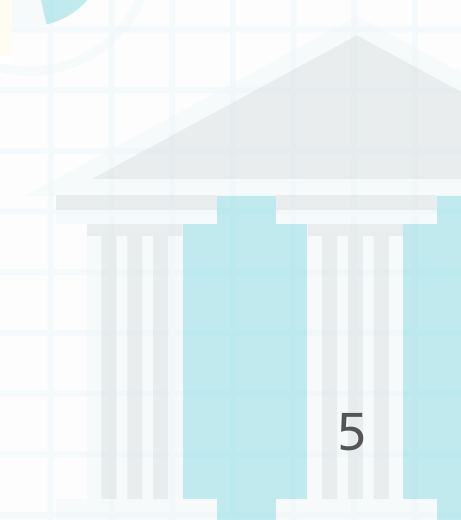
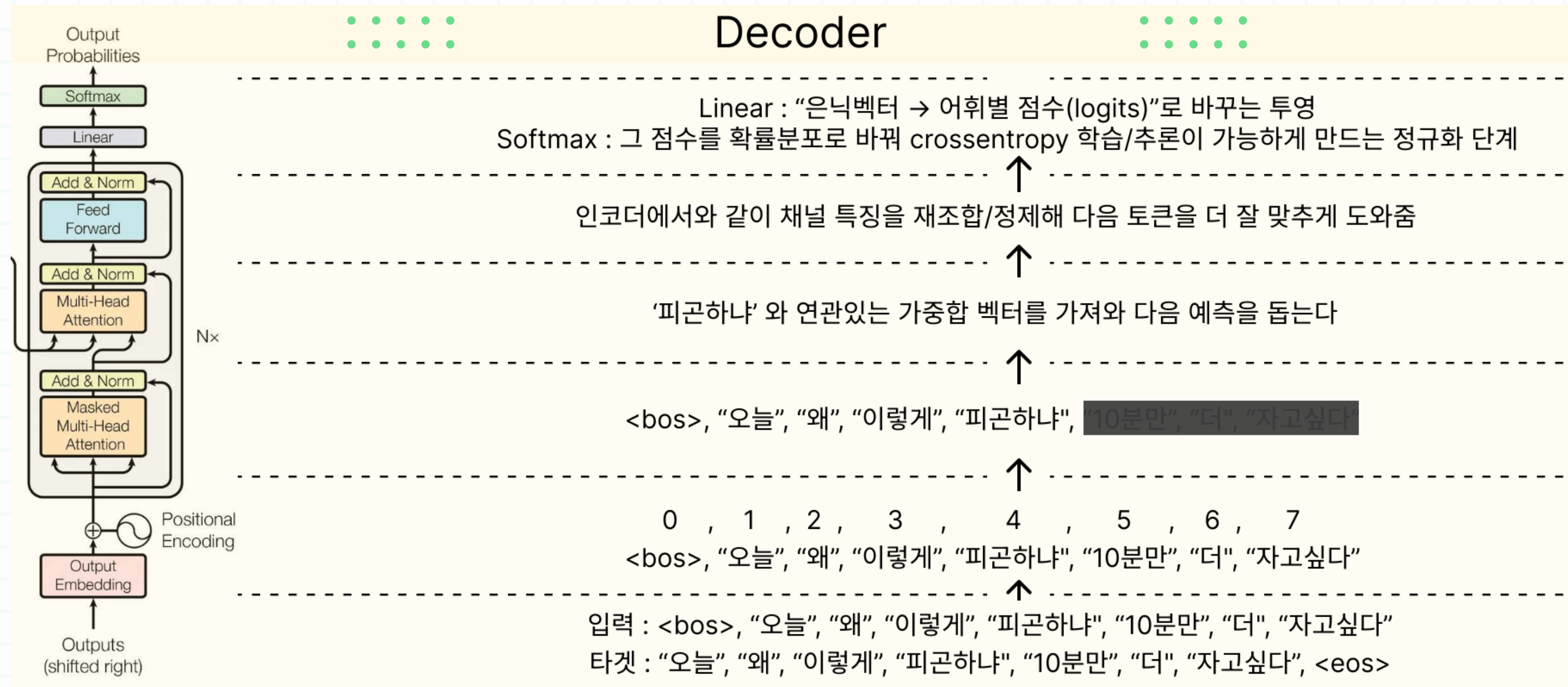
0 , 1 , 2 , 3 , 4 , 5 , 6
“오늘”, “왜”, “이렇게”, “피곤하냐”, “10분만”, “더”, “자고싶다”

‘ 피곤하냐 ’ : [0.9, 0.6, 0.2, 0.8]

모델 차원 : 4

오늘 왜 이렇게 피곤하냐 10분만 더 자고싶다







Attention



Scaled dot-product Attention

1) 개념

q-k 내적을 $\sqrt{d_k}$ 로 스케일 $\rightarrow$ softmax $\rightarrow$ v 가중 평균

2) 효과

기울기 소실 방지: 차원 d가 커질수록 q·k의 분산이 커져 점수가 쉽게 과도한 크기가 됨

학습 안정성: 점수 분포의 스케일을 안정화

temperature 조절: 스케일이 작을수록 소프트맥스가 덜 날카로움 $\rightarrow$ 과신 예방(softmax 포화 예방)

$$\text{Attention}(Q, K, V)$$

$$= \text{softmax}\left(\underbrace{\frac{QK^T}{\sqrt{d_k}} + M}_{\text{Scale}}\right) V$$

$$\frac{e^{(Z_i/T)}}{\sum_j^n e^{(Z_j/T)}} = \text{softmax T(temperature)처럼 동작}$$



	T	p(0)	p(4)	p(8)	p(16)	p(20)
	1	0.000000	0.000000	0.000006	0.017986	0.982008
차갑게	0.5	0.000000	0.000000	0.000000	0.000335	0.999665
	2	0.000040	0.000295	0.002178	0.118903	0.878584
	5	0.011454	0.025490	0.056730	0.280984	0.625342
뜨겁게	10	0.058619	0.087449	0.130458	0.290339	0.433136

ex) [0, 4, 8, 16, 20]



Retention



Symmetric Power Retention

1) 개념

attention을 커널화해서 State에 담아두고 현재 q로 State를 갱신하는 구조

2) 효과

복잡도: $O(N^2) \rightarrow O(N)$, per-token 비용은 $O(1)$

메모리: 과거 KV 전체 대신, 고정된 크기의 state만 유지

표현력: RNN처럼 순차적이면서, 어텐션처럼 장기 의존성 표현

실용성: 긴 컨텍스트·스트리밍·온디바이스에서 훨씬 유리

$$\sum_{j \leq i} \langle q_i, k_j \rangle^p v_j = \phi(q_i)^\top S_i$$

$$S_t = f(S_{t-1}, \phi(k_t), v_t, \text{gate}_t),$$

$$S_i = \sum_{j \leq i} \phi(k_j) v_j^\top,$$

$$y_t = g(\phi(q_t), S_t).$$



Attention



B : batch size

N : sequence length

H : number of heads

C : head dimension

$D = H \cdot C$: model dim

S : retention state dim per head

Input: $x \in \mathbb{R}^{B \times N \times D}$

1) Q,K,V projection

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V,$$

$$W_Q, W_K, W_V \in \mathbb{R}^{D \times D}$$

$$\Rightarrow \text{cost}_{QKV} = 3 \cdot B \cdot N \cdot D \cdot D = \mathcal{O}(BND^2)$$

2) Reshape to heads

$$Q, K, V : [B, N, D] \rightarrow [B, H, N, C], \quad D = HC$$

3) Score $QK^\top$

$$Q \in \mathbb{R}^{B \times H \times N \times C}, \quad K \in \mathbb{R}^{B \times H \times N \times C}$$

$$QK^\top : [B, H, N, C] [B, H, C, N] \rightarrow [B, H, N, N]$$

$$\Rightarrow \text{cost}_{QK^\top} = B \cdot H \cdot N \cdot N \cdot C = \mathcal{O}(BN^2HC) = \mathcal{O}(BN^2D)$$

4) Attn weights AV

$$A \in \mathbb{R}^{B \times H \times N \times N}, \quad V \in \mathbb{R}^{B \times H \times N \times C}$$

$$AV : [B, H, N, N] [B, H, N, C] \rightarrow [B, H, N, C]$$

$$\Rightarrow \text{cost}_{AV} = B \cdot H \cdot N \cdot N \cdot C = \mathcal{O}(BN^2D)$$

5) Head concat + output projection

$$[B, H, N, C] \rightarrow [B, N, HC] = [B, N, D] \quad (\text{concat})$$

$$[B, N, D] [D, D] \rightarrow [B, N, D]$$

$$\Rightarrow \text{cost}_{\text{out}} = B \cdot N \cdot D \cdot D = \mathcal{O}(BND^2)$$

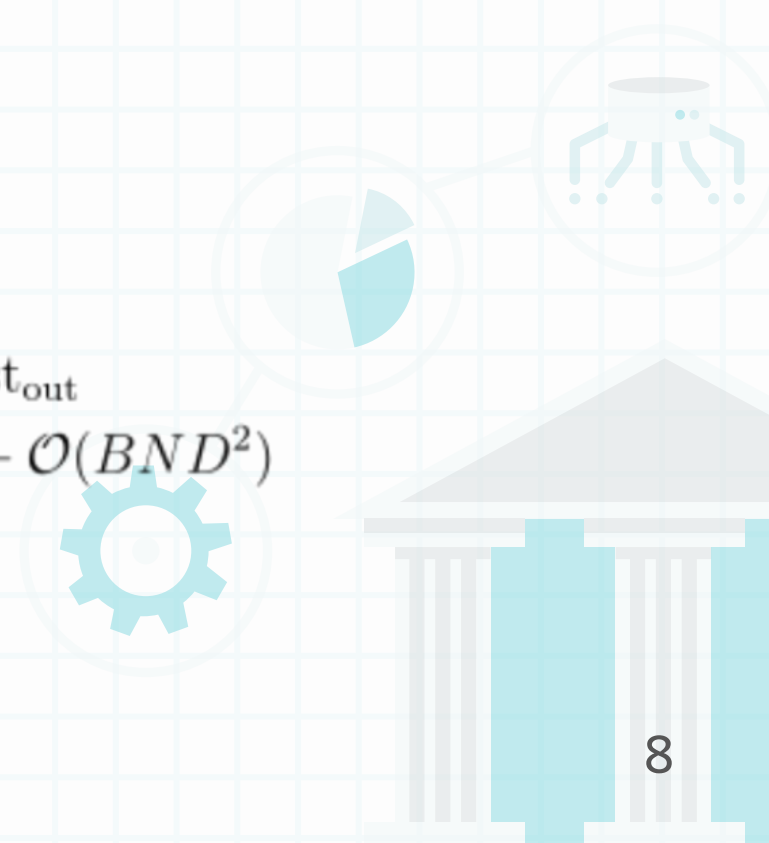
6) Total attention cost

$$\text{cost}_{\text{Attn}} = \text{cost}_{QKV} + \text{cost}_{QK^\top} + \text{cost}_{AV} + \text{cost}_{\text{out}}$$

$$= \mathcal{O}(BND^2) + \mathcal{O}(BN^2D) + \mathcal{O}(BN^2D) + \mathcal{O}(BND^2)$$

$$= \mathcal{O}(BND^2 + BN^2D)$$

$$\approx \mathcal{O}(BN^2D)$$





Retention



B : batch size

N : sequence length

H : number of heads

C : head dimension

$D = H \cdot C$: model dim

S : retention state dim per head

Per-head state: $S_n \in \mathbb{R}^{B \times H \times S \times C}$

1) Power embedding $\phi(q_n), \phi(k_n)$

$$q_n, k_n \in \mathbb{R}^{B \times H \times C}$$

$$\phi(q_n), \phi(k_n) \in \mathbb{R}^{B \times H \times S}$$

$$\Rightarrow \text{cost}_\phi = \mathcal{O}(B \cdot H \cdot C \cdot S) = \mathcal{O}(BDS)$$

2) State update $S_{n-1} \rightarrow S_n$

$$S_{n-1}, S_n \in \mathbb{R}^{B \times H \times S \times C}$$

$$\Rightarrow \text{cost}_{\text{state}} = \mathcal{O}(B \cdot H \cdot S \cdot C) = \mathcal{O}(BDS)$$

3) Output $y_n = g(\phi(q_n), S_n)$

$$y_n \in \mathbb{R}^{B \times H \times C}$$

$$\Rightarrow \text{cost}_{\text{out}} = \mathcal{O}(B \cdot H \cdot S \cdot C) = \mathcal{O}(BDS)$$

4) Per-token cost (one layer)

$$\begin{aligned} \text{cost}_{\text{Ret,token}} &= \text{cost}_\phi + \text{cost}_{\text{state}} + \text{cost}_{\text{out}} \\ &= \mathcal{O}(BDS) \end{aligned}$$

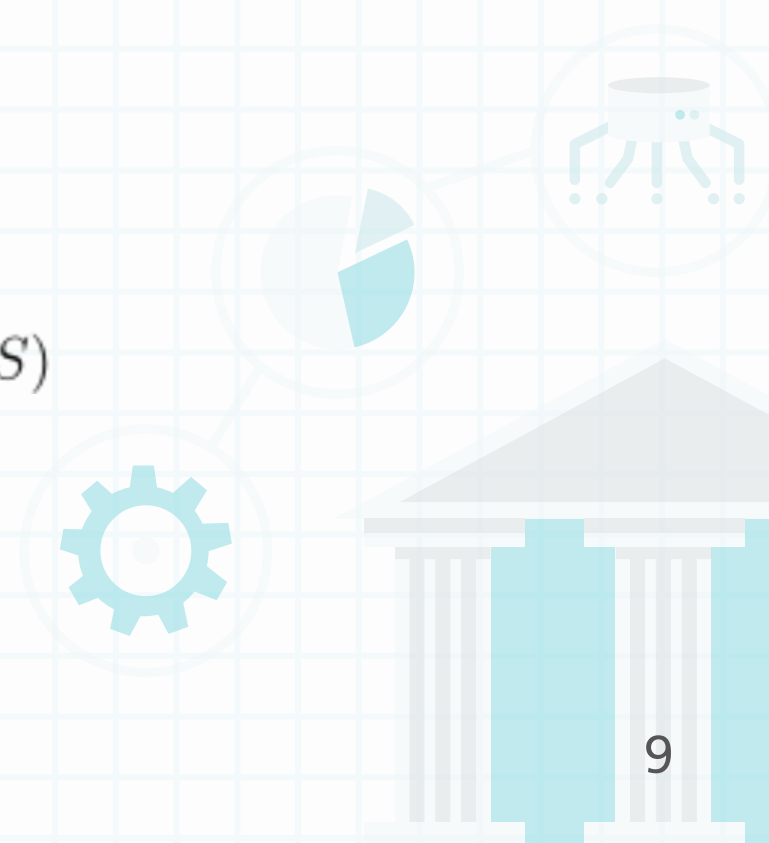
5) Full sequence of length N

$$\begin{aligned} \text{cost}_{\text{Ret,seq}} &= \sum_{n=1}^N \text{cost}_{\text{Ret,token}} \\ &= N \cdot \mathcal{O}(BDS) = \mathcal{O}(BNDS) \end{aligned}$$

6) State memory (per layer)

$$S_n \in \mathbb{R}^{B \times H \times S \times C}$$

$$\Rightarrow \text{mem}_{\text{Ret}} = \Theta(B \cdot H \cdot S \cdot C) = \Theta(BDS)$$





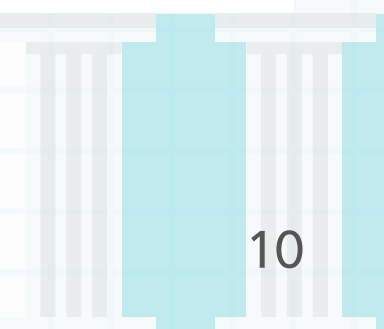
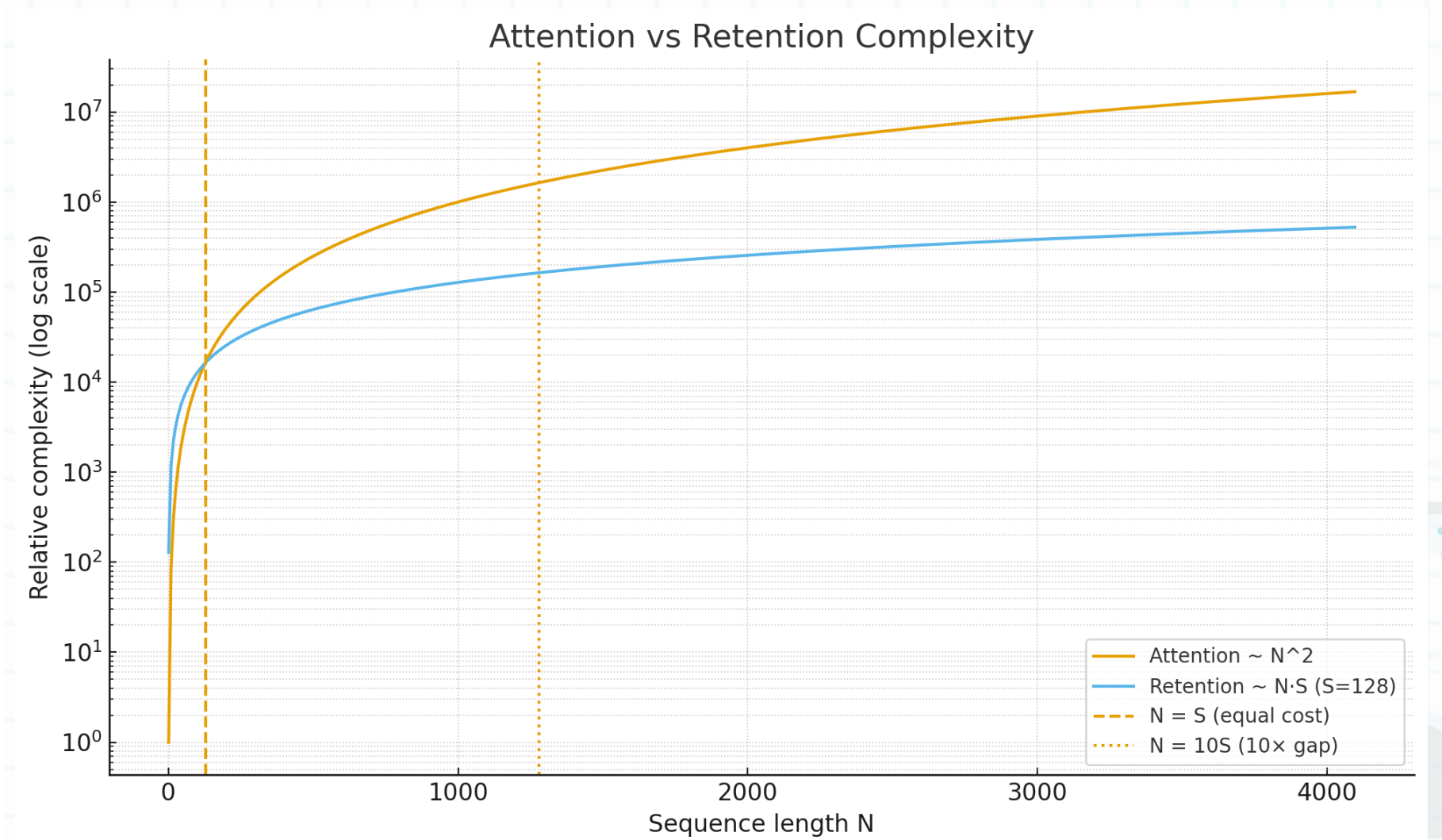
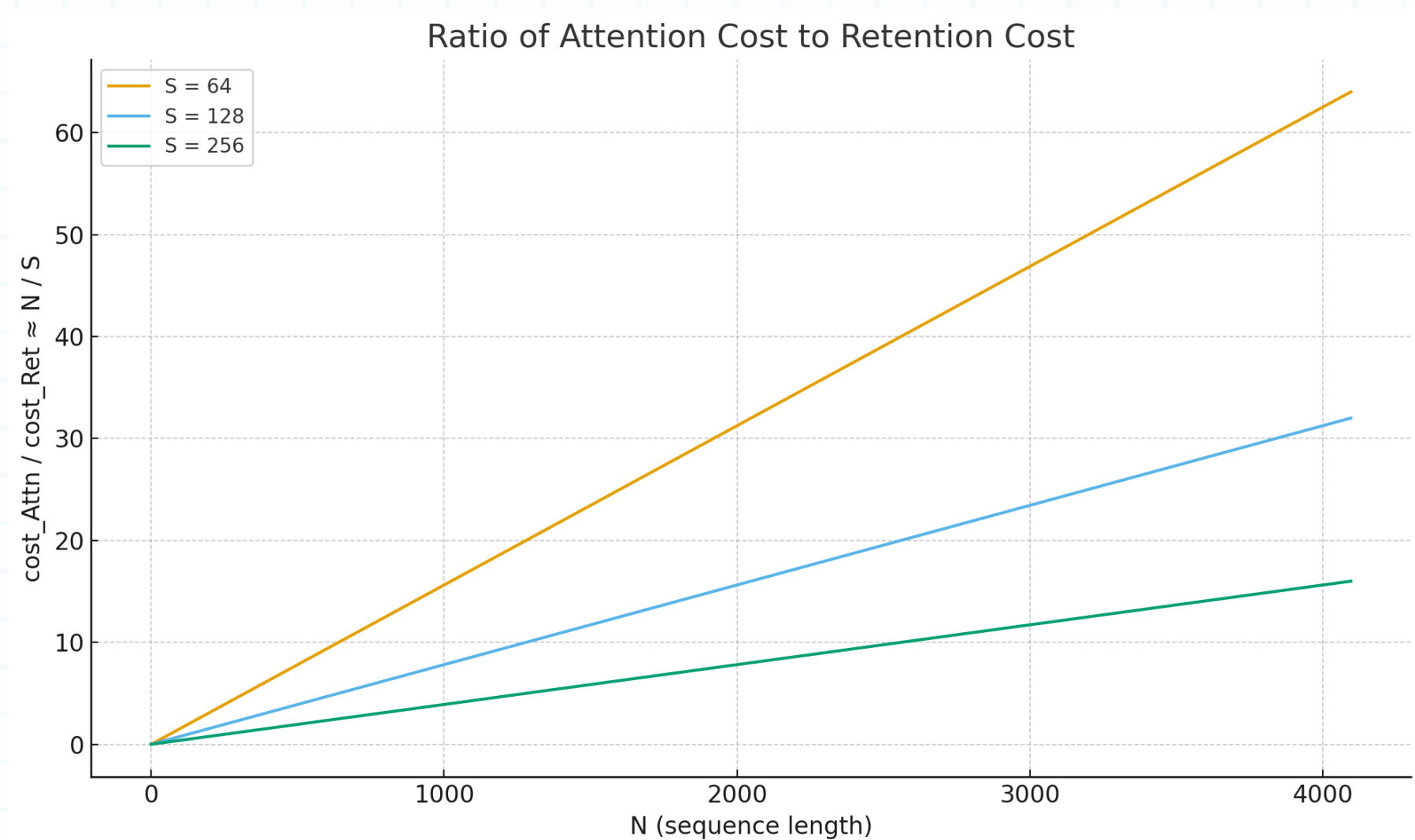
Compare



$$\text{cost}_{\text{Attn}} \approx \mathcal{O}(BN^2D)$$

$$\text{cost}_{\text{Ret,seq}} = \mathcal{O}(BND S)$$

$$\frac{\text{cost}_{\text{Attn}}}{\text{cost}_{\text{Ret,seq}}} = \mathcal{O}\left(\frac{BN^2D}{BND S}\right) = \mathcal{O}\left(\frac{N}{S}\right)$$





Code



```
def scaled_dot_product_attention(self, q, k, v, mask=None):
    # q, k, v: [B, T_x, H, D_head] after projection & reshaping
    # Returning tensor: [B, H, T_q, D_head]
    d_k = k.size(-1)
    k_transpose = k.transpose(3, 2) # [B, H, T_k, D_head] -> [B, H, D_head, T_k]
    # attn_logits: [B, H, T_q, T_k]
    attn_logits = torch.matmul(q, k_transpose) / math.sqrt(d_k)

    if mask is not None:
        if mask.dtype == torch.bool:
            # mask assumed broadcastable to [B, H, T_q, T_k]; True entries masked
            attn_logits = attn_logits.masked_fill(mask, -1e9)
        else:
            attn_logits = attn_logits + mask

    attn = F.softmax(attn_logits, dim=-1) # attention weights over keys
    output = torch.matmul(attn, v) # -> [B, H, T_q, D_head]
    return output
```

```
def symmetric_power2(self, v: torch.Tensor) -> torch.Tensor:
    """
    Compute degree-2 symmetric power features for vector(s) v.

    Args:
        v: Tensor with shape [..., D] (here D == d_head).

    Returns:
        Tensor with shape [..., L] where L = D * (D + 1) / 2 (upper-triangular entries).
    """
    *prefix, D = v.shape
    v_flat = v.reshape(-1, D) # [N, D]

    # outer = v v^T (full D x D)
    outer = v_flat.unsqueeze(2) * v_flat.unsqueeze(1) # [N, D, D]

    # 윗삼각형 인덱스 (i <= j)
    idx = torch.triu_indices(D, D, offset=0, device=v.device) # [2, L]
    i_idx, j_idx = idx[0], idx[1]
    L = idx.shape[1]

    # sqrt(count) 가중치: diag -> 1, off-diag -> sqrt(2)
    diag_mask = (i_idx == j_idx)
    weights = torch.where(
        diag_mask,
        torch.ones(L, device=v.device, dtype=v.dtype),
        torch.full((L,), math.sqrt(2.0), device=v.device, dtype=v.dtype),
    ) # [L]

    # 윗삼각형만 뽑고 가중치 곱하기
    tri = outer[:, i_idx, j_idx] # [N, L] upper-triangular entries
    tri = tri * weights # [N, L]

    # 원래 배치 차원으로 복원
    out = tri.reshape(*prefix, L) # [..., L]
    return out
```



Code



Attention

```
=== Attention Transformer ===
Trainable params: 14,904,200
=====
Layer (type:depth-idx)      Param #      Output Shape
=====
AttentionTransformer        --           [2, 64, 5000]
├─Embedding: 1-1            1,280,000    [2, 64, 256]
├─ModuleList: 1-2          --           --
│   └─AttentionEncoder: 2-1  789,760      [2, 64, 256]
│   └─AttentionEncoder: 2-2  789,760      [2, 64, 256]
│   └─AttentionEncoder: 2-3  789,760      [2, 64, 256]
│   └─AttentionEncoder: 2-4  789,760      [2, 64, 256]
│   └─AttentionEncoder: 2-5  789,760      [2, 64, 256]
│   └─AttentionEncoder: 2-6  789,760      [2, 64, 256]
├─Embedding: 1-3            1,280,000    [2, 64, 256]
├─ModuleList: 1-4          --           --
│   └─AttentionDecoder: 2-7  1,053,440    [2, 64, 256]
│   └─AttentionDecoder: 2-8  1,053,440    [2, 64, 256]
│   └─AttentionDecoder: 2-9  1,053,440    [2, 64, 256]
│   └─AttentionDecoder: 2-10 1,053,440    [2, 64, 256]
│   └─AttentionDecoder: 2-11 1,053,440    [2, 64, 256]
│   └─AttentionDecoder: 2-12 1,053,440    [2, 64, 256]
└─Linear: 1-5               1,285,000    [2, 64, 5000]
=====
Total params: 14,904,200
Trainable params: 14,904,200
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 29.81
=====
```

Retention

```
=== Retention Transformer ===
Trainable params: 14,941,208
=====
Layer (type:depth-idx)      Param #      Output Shape
=====
RetentionTransformer        --           [2, 64, 5000]
├─Embedding: 1-1            1,280,000    [2, 64, 256]
├─ModuleList: 1-2          --           --
│   └─RetentionEncoder: 2-1  791,816      [2, 64, 256]
│   └─RetentionEncoder: 2-2  791,816      [2, 64, 256]
│   └─RetentionEncoder: 2-3  791,816      [2, 64, 256]
│   └─RetentionEncoder: 2-4  791,816      [2, 64, 256]
│   └─RetentionEncoder: 2-5  791,816      [2, 64, 256]
│   └─RetentionEncoder: 2-6  791,816      [2, 64, 256]
├─Embedding: 1-3            1,280,000    [2, 64, 256]
├─ModuleList: 1-4          --           --
│   └─RetentionDecoder: 2-7  1,057,552    [2, 64, 256]
│   └─RetentionDecoder: 2-8  1,057,552    [2, 64, 256]
│   └─RetentionDecoder: 2-9  1,057,552    [2, 64, 256]
│   └─RetentionDecoder: 2-10 1,057,552    [2, 64, 256]
│   └─RetentionDecoder: 2-11 1,057,552    [2, 64, 256]
│   └─RetentionDecoder: 2-12 1,057,552    [2, 64, 256]
└─Linear: 1-5               1,285,000    [2, 64, 5000]
=====
Total params: 14,941,208
Trainable params: 14,941,208
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 29.88
=====
```



Code



Attention

module	#parameters or shape	#flops
model	14.904M	1.654G
input_data_embed	1.28M	0
input_data_embed.weight	(5000, 256)	
output_data_embed	1.28M	0
output_data_embed.weight	(5000, 256)	
last_linear_layer	1.285M	0.164G
last_linear_layer.weight	(5000, 256)	
last_linear_layer.bias	(5000,)	
Encoders	4.739M	0.631G
Encoders.0	0.79M	0.105G
Encoders.1	0.79M	0.105G
Encoders.2	0.79M	0.105G
Encoders.3	0.79M	0.105G
Encoders.4	0.79M	0.105G
Encoders.5	0.79M	0.105G
Decoders	6.321M	0.859G
Decoders.0	1.053M	0.143G
Decoders.1	1.053M	0.143G
Decoders.2	1.053M	0.143G
Decoders.3	1.053M	0.143G
Decoders.4	1.053M	0.143G
Decoders.5	1.053M	0.143G

Retention

module	#parameters or shape	#flops
model	14.941M	2.06G
input_data_embed	1.28M	0
input_data_embed.weight	(5000, 256)	
output_data_embed	1.28M	0
output_data_embed.weight	(5000, 256)	
last_linear_layer	1.285M	0.164G
last_linear_layer.weight	(5000, 256)	
last_linear_layer.bias	(5000,)	
Encoders	4.751M	0.766G
Encoders.0	0.792M	0.128G
Encoders.1	0.792M	0.128G
Encoders.2	0.792M	0.128G
Encoders.3	0.792M	0.128G
Encoders.4	0.792M	0.128G
Encoders.5	0.792M	0.128G
Decoders	6.345M	1.129G
Decoders.0	1.058M	0.188G
Decoders.1	1.058M	0.188G
Decoders.2	1.058M	0.188G
Decoders.3	1.058M	0.188G
Decoders.4	1.058M	0.188G
Decoders.5	1.058M	0.188G



결론



한계

1) 정보 부족

최근에 공개된 모델이기 때문에 정보가 많이 없음

2) 환경 제약

Manifest AI가 공개한 소스코드는 특정 환경에서만 작동하도록 설계되어 있음

<Linux 운영체제 - CUDA 12.4 버전 - Python 3.12 이상>

3) CUDA 커널 최적화가 특히 중요한 것으로 보임

```
## No Chunk - 연산량 / 시간 비교
- Attention params: 14,904,200
- Retention params: 14,941,208
- Attention FLOPs: 1.654 G
- Retention FLOPs: 2.060 G
- Attention avg forward time: 22.18 ms
- Retention avg forward time: 1314.63 ms
```

(위 결과는 CUDA 커널 최적화를 진행하지 않은 상태임)
최적화를 진행하지 않으면 레이턴시가 크게 증가함

4) 테스트 환경 구성 제약

복잡도가 낮아 속도에 장점이 있을 것으로 보이나 State의 차원보다 토큰 수가 더 많을 때 속도적 장점이 드러날 것으로 보임

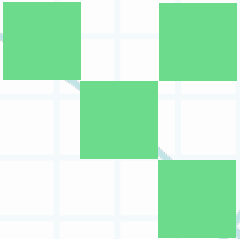
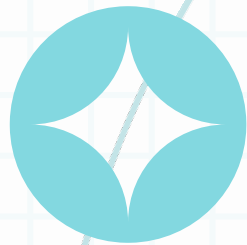
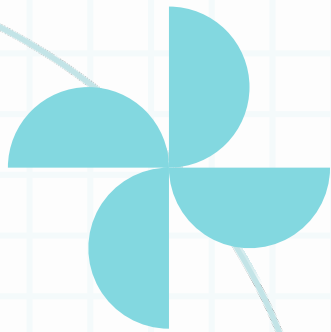
$$S = \binom{64 + 2 - 1}{2} = \binom{65}{2} = 2080$$

$$S = \binom{64 + 3 - 1}{3} = \binom{66}{3} = 45760$$

p=2일 때 $N > 2080$ / p=3일 때 $N > 45760$

굉장히 큰 시퀀스를 처리할 때 속도적 장점이 있을 것임
일반적인 사용에는 Attention이 더 효율적임





감사합니다

지피다 | 연구원 김재현 | E. jh.kim@zipida.com | T. 010-4744-0949